



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Agentic AI

Michele Colajanni, Claudio Zanasi

Architetture Avanzate di Agenti

Pattern di Sviluppo, Multi-Agente e Framework

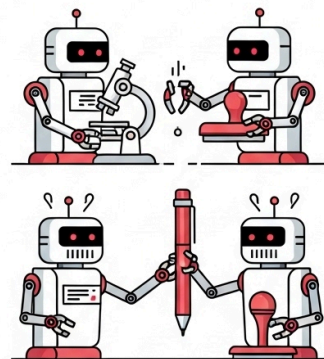
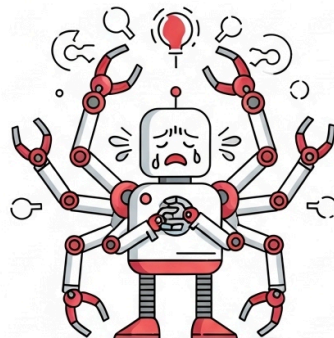
Singolo Agente vs. Multi-Agente

Singolo Agente: Un solo LLM con un prompt di sistema enorme e accesso a tutti i tool.

- **Pro:** Facile da implementare all'inizio.
- **Contro:** Soffre di "Lost in the Middle", difficile debug se fallisce un task complesso.

Multi-Agente: Più agenti, ognuno con un prompt specifico, un ruolo chiaro e un set ridotto di tool.

- **Pro:** Altamente specializzato, modulare, facile da testare e scalabile.
- **Contro:** Maggiore complessità architetturale e latenza (più chiamate LLM).



Perché Scomporre? Il Potere della Specializzazione

Scomporre un problema complesso per un team di agenti risolve i limiti fondamentali degli LLM:

1. **Riduzione del Contesto:** Ogni agente riceve solo le informazioni rilevanti per il suo task, riducendo le allucinazioni.
2. **Prompt Focalizzati:** Un prompt che dice "Sei un revisore legale severo" funziona molto meglio di un prompt che dice "Cerca i dati, scrivi il testo e poi fai finta di essere un revisore legale".
3. **Separazione delle Responsabilità (SoC):** Se la ricerca fallisce, modifichi l'Agente Ricercatore senza rischiare di rompere l'Agente Scrittore.

I 3 Modelli Architettureali Principali

Quando passiamo al Multi-Agente, dobbiamo decidere *come* questi agenti interagiscono. Esistono tre pattern fondamentali:

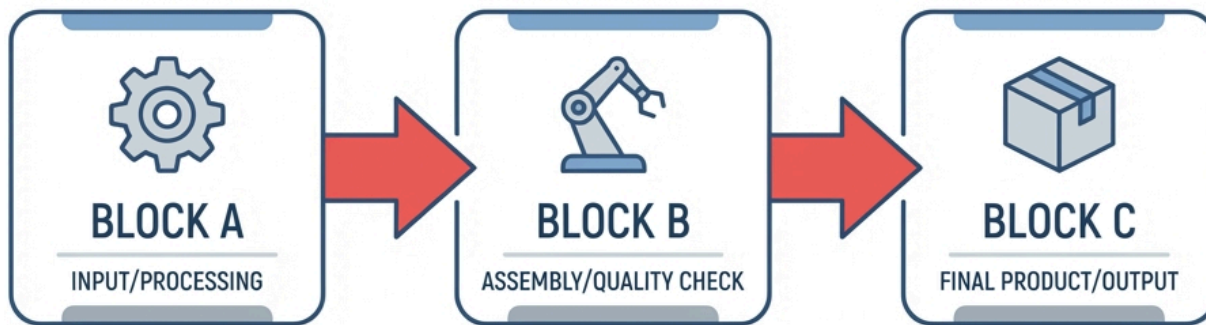
1. **Sequenziale (Pipeline):** Il lavoro passa da un agente all'altro in linea retta.
2. **Gerarchico (Manager/Worker):** Un agente "Capo" pianifica e delega i task ai subordinati.
3. **Collaborativo (Consenso):** Gli agenti discutono tra loro in modo paritetico per arrivare a una soluzione condivisa.

1. Modello Sequenziale (Pipeline)

È il modello più semplice e prevedibile. L'output dell'Agente A diventa l'input dell'Agente B.

Flusso Tipico: Ricercatore -> Scrittore -> Traduttore

Quando usarlo: Per processi aziendali lineari, ben definiti e ripetitivi, dove non c'è bisogno di tornare indietro o prendere decisioni complesse sul routing.



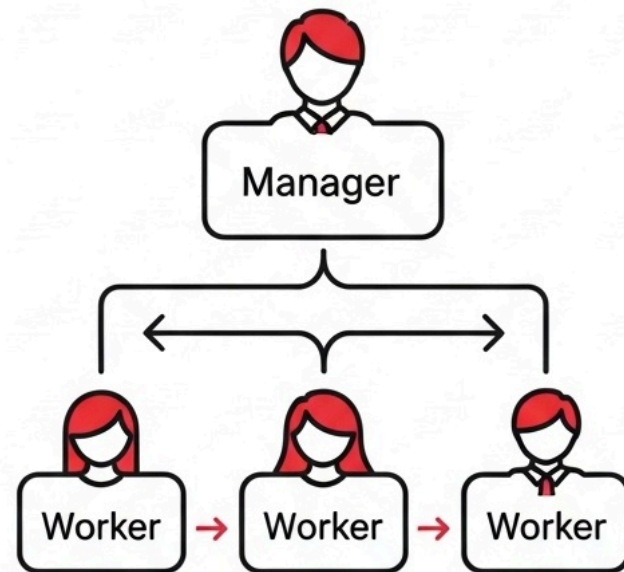
LINEAR PRODUCTION FLOW - ASSEMBLY LINE MODEL

2. Modello Gerarchico (Manager/Worker)

Introduce un **Agente Supervisore (Manager)**. Il Manager non esegue il lavoro pratico, ma:

1. Riceve la richiesta dell'utente.
2. Crea un piano.
3. Delega i sotto-task agli agenti specializzati (Worker).
4. Raccoglie i risultati e formula la risposta finale.

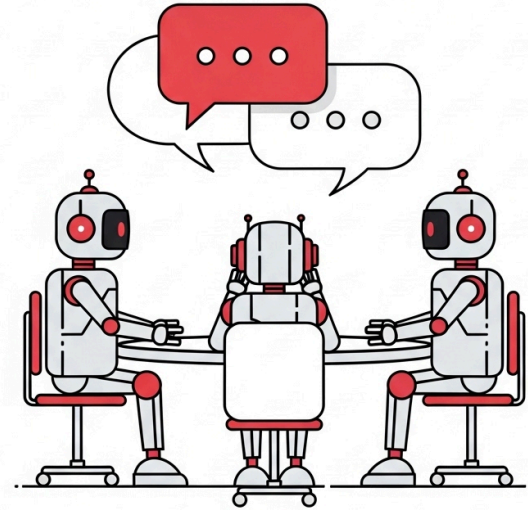
Quando usarlo: Per task complessi e ambigui dove il flusso di lavoro non è noto a priori e richiede pianificazione dinamica.



3. Modello Collaborativo (Consenso)

Gli agenti operano in una "chat room" condivisa. Non c'è un capo rigido. Discutono, propongono soluzioni e si criticano a vicenda finché non raggiungono un accordo.

Quando usarlo: Per task creativi, brainstorming, o problemi aperti che richiedono molteplici punti di vista (es. un agente "Sviluppatore" e un agente "Tester di Sicurezza" che discutono su un pezzo di codice).



Lo Stato nei Sistemi Multi-Agente

Come comunicano gli agenti? Raramente si passano messaggi diretti (A parla a B).

Utilizzano lo **Stato** come una **Lavagna Condivisa (Shared Whiteboard)**.

1. Il Manager scrive il piano sulla lavagna.
2. Il Ricercatore legge il piano, esegue la ricerca e appunta i risultati sulla lavagna.
3. Lo Scrittore legge i risultati dalla lavagna e scrive la bozza.

Questo disaccoppia gli agenti: nessuno deve sapere chi ha scritto i dati, basta che siano presenti nello Stato.

Criteri di Scelta dell'Architettura

Come scegliere l'architettura giusta per il tuo progetto?

Criterio	Sequenziale	Gerarchico	Collaborativo
Complessità Task	Bassa / Ripetitiva	Alta / Dinamica	Molto Alta / Creativa
Prevedibilità	Massima	Media	Bassa
Costo (Token)	Basso	Medio-Alto	Molto Alto
Facilità di Debug	Molto Facile	Media	Difficile

Regola d'oro: Inizia sempre con un singolo agente. Se fallisce, passa a una pipeline sequenziale. Usa gerarchie e collaborazioni solo se strettamente necessario.

Pattern Collaborativi e di Riflessione

Orchestrazione Gerarchica in Pratica

Nel pattern Gerarchico, il **Manager** è il cuore del sistema. Spesso è implementato non come un agente ReAct completo, ma come un **LLM Router** (come visto precedentemente).

Il suo prompt è strutturato per prendere decisioni di routing: *"Sei un supervisore. Hai a disposizione l'Agente Ricercatore e l'Agente Matematico. Analizza la richiesta e decidi a chi passare il turno, oppure rispondi 'FINISH' se il lavoro è completo."*

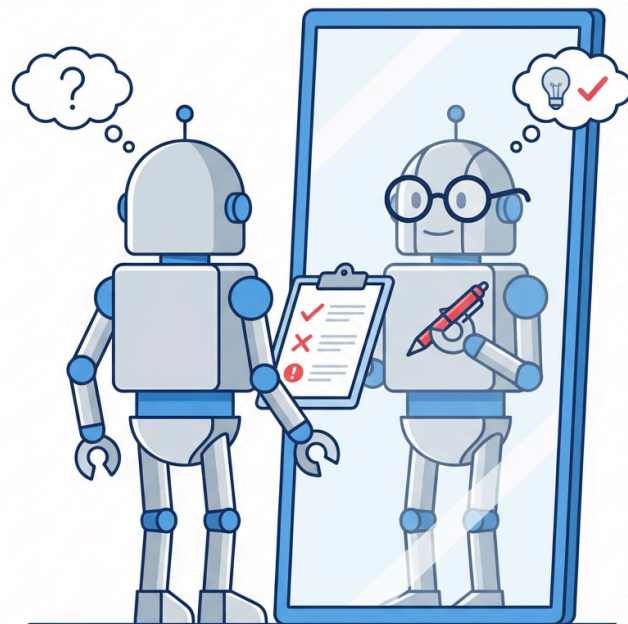
Questo pattern permette di scalare all'infinito aggiungendo nuovi "Worker" sotto il Manager.

Oltre ReAct: Il Pattern di Riflessione

Il pattern ReAct (Reason-Act) è ottimo per *fare* cose, ma è debole nel *valutare* la qualità di ciò che ha fatto.

Il **Pattern di Riflessione (Reflection)** risolve questo problema separando la fase di generazione da quella di critica.

Invece di chiedere all'LLM di scrivere un testo perfetto al primo colpo, gli chiediamo di scrivere una bozza, e poi gli chiediamo di **criticare la sua stessa bozza**.



Il Ciclo Plan-Generate-Reflect-Refine

Questo è il workflow definitivo per massimizzare la qualità dell'output di un LLM:

1. **Plan:** L'agente crea una scaletta di ciò che deve fare.
2. **Generate:** L'agente produce una prima bozza (es. un blocco di codice o un articolo).
3. **Reflect:** L'agente (o un agente diverso) analizza la bozza contro i requisiti iniziali, cercando errori, omissioni o difetti di stile. Produce una lista di critiche.
4. **Refine:** L'agente prende la bozza originale e la lista di critiche, e genera una versione migliorata.

Il ciclo 3 e 4 può ripetersi N volte finché la critica non è "Pass".

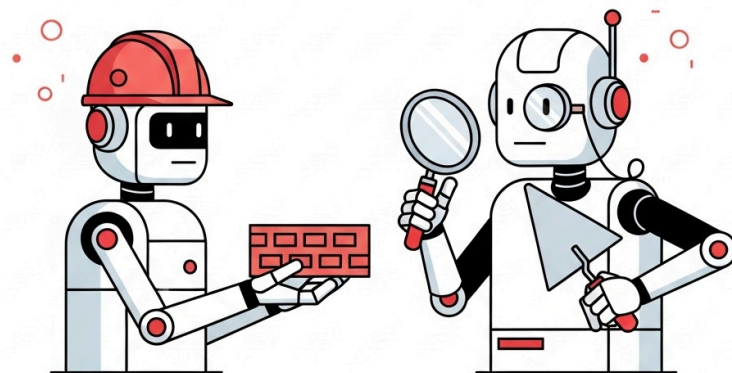
L'Agente Critico (Validazione Interna)

Nel Multi-Agente, la Riflessione dà il meglio di sé quando separiamo i ruoli.

Introduciamo l'**Agente Critico (o Revisore)**.

- Il suo unico scopo è trovare difetti nel lavoro degli altri.
- Ha un prompt di sistema focalizzato sulla pignoleria e sulle regole aziendali.

Esempio: L'Agente Sviluppatore scrive il codice. L'Agente QA (Critico) lo legge, scrive i test e, se falliscono, rimanda il codice allo Sviluppatore con il log degli errori.

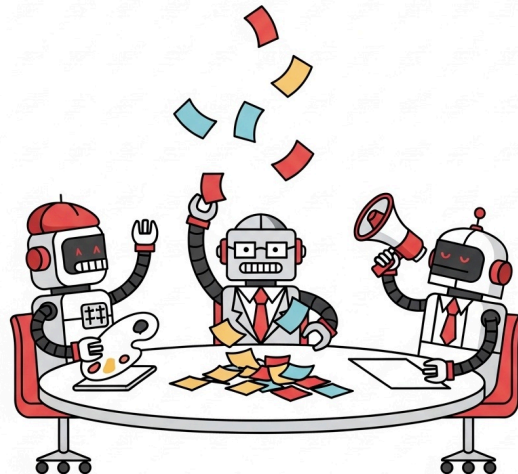


Pattern di Brainstorming (Peer-to-Peer)

Oltre alla gerarchia, gli agenti possono collaborare in modo paritetico per stimolare la creatività.

- **Divergenza:** Si istanziano più agenti con "Personas" diverse (es. Agente Creativo, Agente Analitico, Agente Finanziario) e si chiede a ciascuno di proporre soluzioni indipendenti allo stesso problema.
- **Convergenza:** Dopo la fase di generazione, gli agenti leggono le idee degli altri e le fondono in una lista finale condivisa.

Perché funziona: Gli LLM tendono a convergere rapidamente sulla risposta più "media" o probabile. Forzare prospettive diverse tramite prompt separati garantisce un'esplorazione più ampia dello spazio delle soluzioni.



Pattern Avversariale (Debate / Red Teaming)

Ispirato alla sicurezza informatica (Red Team vs Blue Team) e al sistema legale.

- **Agente Proponente:** Crea una soluzione (es. un'architettura software o un contratto).
- **Agente Avversario:** Ha l'obiettivo esplicito di attaccare la soluzione, trovando falle, edge cases o vulnerabilità.
- **Il Giudice:** Un terzo agente (o l'utente umano) valuta il dibattito e decreta la versione finale.

Perché funziona: Gli LLM sono spesso "sycophants" (tendono a dare ragione all'utente o a confermare le proprie ipotesi). Metterli in competizione forza il modello a esplorare i propri punti ciechi.

Ideale per: Code Review, Analisi Legale, Valutazione dei Rischi.

Pattern Ensemble (Votazione e Maggioranza)

Basato sul principio della "Saggezza della Folla" per massimizzare l'affidabilità.

Invece di far risolvere un problema complesso a un solo agente, si istanziano **N agenti identici** (magari con temperature diverse) e si fa risolvere loro lo stesso problema in parallelo.

Un "Agente Giudice" (o un semplice script Python) raccoglie le N risposte e sceglie la più frequente (**Majority Vote**) o sintetizza la migliore.

Pro: Riduce drasticamente le allucinazioni. Se 4 agenti su 5 danno la stessa risposta matematica o estraggono lo stesso dato, la confidenza è altissima.

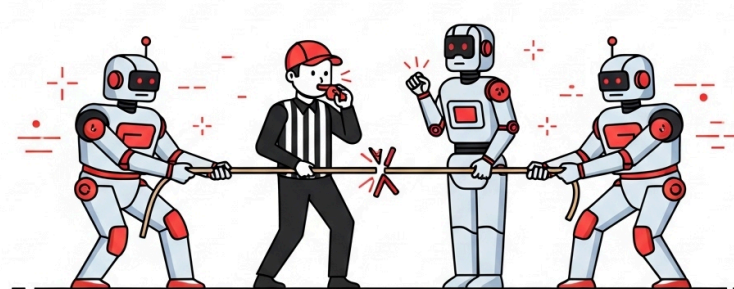
Contro: Il costo in token e la latenza vengono moltiplicati per N. Da usare solo per decisioni critiche dove l'accuratezza è più importante del budget.

Gestione dei Conflitti e Deadlock

Cosa succede se due agenti collaborativi non si mettono d'accordo e continuano a discutere all'infinito? Questo è un **Deadlock**.

Strategie di mitigazione:

1. **Max Iterations:** Impostare sempre un limite rigido ai turni di conversazione (es. max 3 scambi per task).
2. **Tie-breaker Umano:** Se si raggiunge il limite, il sistema si ferma e scala all'umano (HITL).
3. **Agente Moderatore:** Un agente con autorità superiore che interviene per forzare una decisione quando rileva un loop circolare nei log della chat.



Progettare per l'Intervento Umano (Human-in-the-Loop)

Cos'è l'Human-in-the-Loop (HITL)?

L'autonomia totale dell'IA è spesso un rischio inaccettabile in contesti aziendali.

Human-in-the-Loop (HITL) significa progettare il sistema in modo che un essere umano sia parte integrante del processo decisionale dell'agente.

Non si tratta di far fare il lavoro all'umano, ma di usare l'umano come **garante della qualità, della sicurezza e dell'allineamento aziendale**.

Lo Spettro dell'Autonomia

L'implementazione dell'IA non è binaria (tutto manuale o tutto automatico). Esiste uno spettro di autonomia:

1. **Copilot (Assistente):** L'umano guida, l'IA suggerisce (es. GitHub Copilot). L'umano ha il controllo totale.
2. **Agente con HITL (Supervisionato):** L'IA guida e pianifica, ma l'umano approva le decisioni chiave o interviene in caso di eccezioni.
3. **Agente Autonomo:** L'IA opera senza alcun intervento umano (es. trading algoritmico ad alta frequenza).

Il design HITL ci permette di muoverci in modo sicuro dal livello 1 al livello 2, mantenendo i vantaggi dell'automazione senza i rischi del livello 3.

Pattern 1: Approvazione (Gatekeeper)

L'agente fa tutto il lavoro preparatorio, ma si ferma prima di eseguire un'azione irreversibile o sensibile (es. inviare un'email a un cliente, eseguire una query `DROP TABLE`, fare un bonifico).

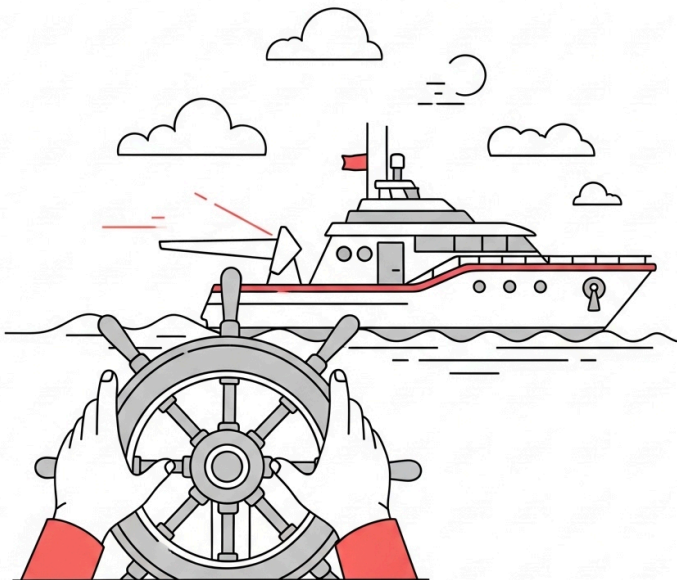
Il sistema sospende l'esecuzione (sfruttando lo State Management visto nel Giorno 3) e invia una notifica all'umano.

L'umano ha due opzioni:

- **Approva:** L'agente riprende l'esecuzione e usa il tool.
- **Rifiuta:** L'agente si ferma o viene reindirizzato a un flusso di errore.



Pattern 2: Guida (Steering)



Più avanzato dell'approvazione binaria (Sì/No). Nel pattern di Guida, l'umano può **correggere la traiettoria** dell'agente mentre sta lavorando.

Se l'agente propone una bozza di email, l'umano non si limita a rifiutarla, ma fornisce un feedback: *"Il tono è troppo formale, rendilo più amichevole e aggiungi uno sconto del 10%"*.

L'agente riceve questo feedback, aggiorna il suo stato (Time Travel / Rewind) e genera una nuova versione.

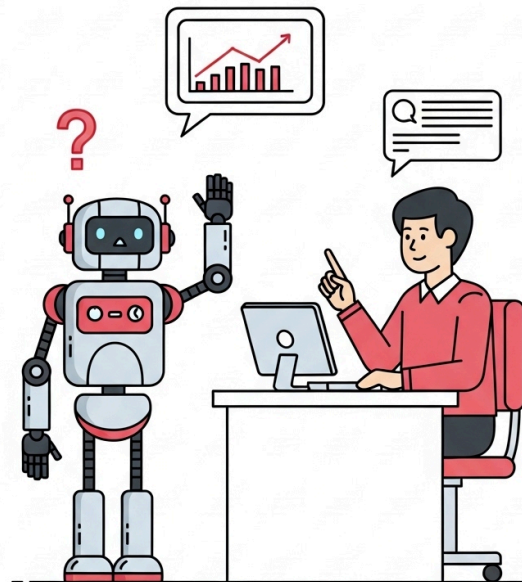
Pattern 3: Escalation (Richiesta di Aiuto)

A differenza dell'approvazione (dove l'agente ha finito e aspetta l'OK), nell'**Escalation** l'agente si rende conto di *non poter finire il lavoro*.

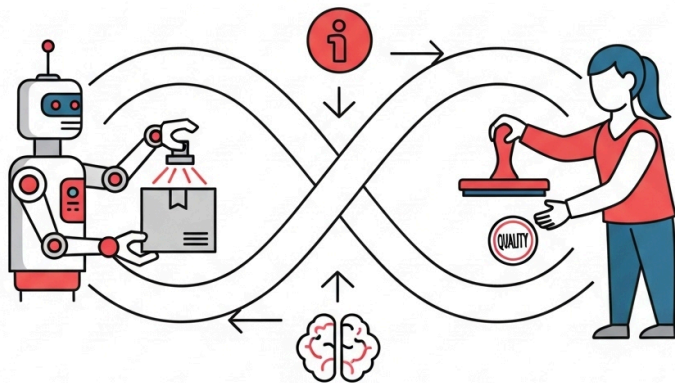
Quando si attiva:

- L'agente non trova le informazioni necessarie nel RAG.
- Un tool restituisce un errore incomprensibile.
- L'agente rileva un'ambiguità nella richiesta originale dell'utente.

Azione: L'agente sospende l'esecuzione e invia un messaggio proattivo.



Pattern 4: Feedback Loop (Miglioramento)



L'HITL non serve solo a salvare la singola transazione, ma a **migliorare il sistema nel tempo**.

Quando l'umano corregge un agente, questa correzione non deve andare persa.

Come implementarlo:

1. L'umano modifica l'output dell'agente.
2. Il sistema salva la coppia (Contesto Originale, Correzione Umana).
3. Questi dati vengono usati per aggiornare il prompt di sistema, arricchire il Vector DB (RAG) o per futuri cicli di Fine-Tuning (RLHF).

Interfacce e UX per l'HITL

Come comunicano fisicamente l'agente e l'umano? La scelta dell'interfaccia è cruciale per non creare colli di bottiglia.

- **Chat Sincrona (es. ChatGPT, Slack, Teams):** Ideale per task rapidi e interazioni discorsive. L'agente "tagga" l'umano nel canale.
- **Notifiche Asincrone (es. Email, Telegram):** Ottime per processi in background di lunga durata. L'agente invia un link con due pulsanti: "Approva" o "Rifiuta".
- **Dashboard Dedicate (Approval Queues):** Per volumi elevati (es. customer service). Una UI web dove gli operatori vedono una lista di task in attesa di revisione, con le motivazioni dell'agente e i dati di contesto affiancati.

Sicurezza e Affidabilità tramite HITL

Implementare HITL non è solo una questione di UX, è una **misura di sicurezza fondamentale**.

- Previene i danni causati dalle allucinazioni.
- Mitiga i rischi di Prompt Injection (se un attaccante forza l'agente a fare un bonifico, l'umano bloccherà l'azione al momento dell'approvazione).
- Crea un "Audit Trail": ogni azione sensibile ha la firma di un operatore umano che l'ha autorizzata.

Nel Laboratorio 1 di oggi pomeriggio, implementeremo esattamente questo: un'approvazione via Telegram prima di inviare un report.

Introduzione ai Framework Agentici

La Classificazione dei Framework

Il mercato offre decine di framework per lo sviluppo di agenti AI. Possiamo classificarli in quattro grandi famiglie in base al loro livello di astrazione e al paradigma di sviluppo:

1. **Orchestrazione a Grafo (Basso Livello):** Focus sul controllo totale del flusso tramite macchine a stati.
2. **Role-Playing / Task-Driven (Alto Livello):** Focus sulla simulazione di team umani e prototipazione rapida.
3. **Code-Driven (Esecuzione Codice):** Focus sull'espressività, dove l'agente scrive script Python invece di usare JSON.
4. **Modulari e Tipizzati (Atomic):** Focus sull'ingegneria del software tradizionale, con input/output rigorosamente tipizzati.

Nelle prossime slide analizzeremo nel dettaglio ciascuna di queste categorie.

1. Orchestrazione a Grafo (Basso Livello)

Esempi: LangGraph, LlamaIndex Workflows.

Come funziona: Definisci esplicitamente il flusso di esecuzione come un grafo (spesso ciclico). Ogni **nodo** rappresenta un'operazione (una chiamata LLM, uno script Python, un tool), e ogni **arco** rappresenta una condizione di routing (es. "Se l'output contiene 'errore', torna al nodo precedente").

Lo stato globale viene passato di nodo in nodo, permettendo un controllo granulare su cosa l'agente "ricorda" in ogni fase.

Pro:

- Controllo totale e flessibilità massima.
- Perfetto per flussi complessi, non standard e altamente personalizzati.
- Ottima gestione dello stato (time-travel).

Contro:

- Curva di apprendimento molto ripida.
- Richiede di scrivere molto codice.
- Difficile da prototipare velocemente rispetto ai framework di alto livello.

2. Role-Playing / Task-Driven (Alto Livello)

Esempi: CrewAI, AutoGen, Swarm.

Come funziona: Il paradigma si basa sulla simulazione di un team umano. Definisci gli agenti assegnando loro un **Ruolo**, un **Obiettivo** e un **Background** (Backstory).

Successivamente, definisci una lista di **Task** e li assegna agli agenti. Il framework si occupa automaticamente dell'orchestrazione (sequenziale o gerarchica) e del passaggio di informazioni tra gli agenti.

Pro:

- Velocità di sviluppo.
- Pattern multi-agente pronti all'uso.
- Codice molto leggibile e accessibile anche a figure non tecniche (es. Prompt Engineers).

Contro:

- Meno flessibili per logiche di routing condizionale complesse.
- Rischio di "black box": a volte è difficile capire *perché* è stata presa una certa decisione.

3. Code-Driven (Esecuzione Codice)

Esempi: smolagents (Hugging Face).

Come funziona: Invece di usare il classico "Function Calling" basato su JSON (dove l'LLM restituisce un JSON con il nome del tool e i parametri), l'agente **scrive ed esegue direttamente codice Python**. L'LLM genera uno script che importa i tool, li esegue, salva i risultati in variabili e implementa logica condizionale (if/else, cicli for).

Pro:

- Estremamente potente ed espressivo.
- Permette di combinare dinamicamente i risultati di più tool in un solo step.
- Sfrutta il fatto che gli LLM sono stati addestrati su codice.

Contro:

- Rischio di errori di sintassi nel codice generato.
- **Sicurezza:** Richiede l'uso di sandbox sicure per evitare l'esecuzione di codice malevolo.

4. Modulari e Tipizzati (Atomic)

Esempi: Atomic Agents.

Come funziona: Abbandona l'idea dei grandi agenti "tuttofare" in favore di piccoli agenti atomici, seguendo il *Single Responsibility Principle* dell'ingegneria del software. Usa librerie come **Pydantic** e **Instructor** per forzare l'LLM a rispettare schemi dati rigidi sia in input che in output. I tool non sono monolitici, ma componenti standalone scaricabili nel progetto.

Pro:

- Codice estremamente pulito, prevedibile e type-safe.
- Facile da testare e da integrare in pipeline software tradizionali.
- Agnostico rispetto al provider.

Contro:

- Richiede di scrivere più codice strutturale rispetto ai framework di alto livello.
- Meno "magico" out-of-the-box per orchestrazioni complesse.

Astrazioni Comuni: Agenti e Task

Indipendentemente dal framework, ritroverai sempre queste astrazioni fondamentali:

- **Agent (L'Attore):** Non è solo un prompt. È un oggetto software che incapsula:
 - L'identità (Role, Goal, Backstory).
 - Il "Motore" (quale specifico LLM usare, es. GPT-4o per ragionamenti complessi, Haiku per task semplici).
 - I limiti (es. `max_iterations` per evitare loop infiniti).
- **Task (Il Compito):** L'unità atomica di lavoro. Definisce:
 - La descrizione chiara di cosa fare.
 - L'**Output Atteso**, spesso forzato tramite schemi rigidi (es. Pydantic models) per garantire che il risultato sia utilizzabile dal software.

Astrazioni Comuni: Tools (Gli Strumenti)

I framework astraggono la complessità del Function Calling.

Tu scrivi una normale funzione Python con type hints e docstrings; il framework si occupa di:

- Tradurla in JSON Schema per l'LLM.
- Eseguirla in modo sicuro.
- Gestire le eccezioni (es. restituendo l'errore all'LLM per fargli riprovare).

Astrazioni Comuni: Memory / Context (Lo Stato)

La memoria permette all'agente di mantenere il contesto durante e tra le esecuzioni:

- **Short-term:** Lo "scratchpad" dove l'agente appunta i risultati intermedi dei tool.
- **Long-term:** Molti framework offrono integrazioni native con Vector DB (es. Chroma, Pinecone) per salvare e recuperare ricordi tra diverse sessioni.
- **Shared State:** L'oggetto condiviso che permette a più agenti di passarsi i risultati in un sistema multi-agente.

Astrazioni Comuni: Orchestrazione (Processi)

Come facciamo a far collaborare gli agenti? I framework forniscono motori di orchestrazione (Processi):

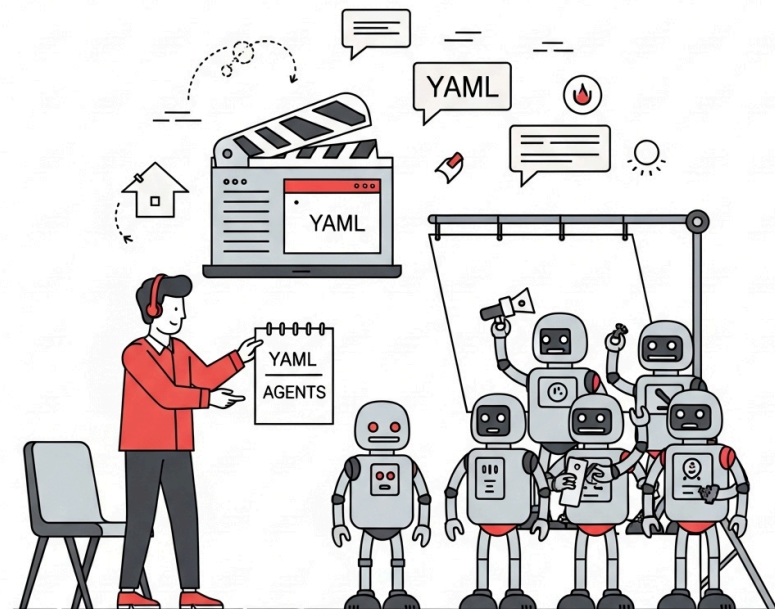
- **Routing Statico (Sequential):** Il framework esegue la lista dei task in ordine rigoroso. L'output del Task 1 viene iniettato automaticamente come contesto nel Task 2.
- **Routing Dinamico (Hierarchical / Supervisor):** Il framework istanzia automaticamente un "Agente Manager" nascosto. Tu gli dai la lista dei task e degli agenti, e lui decide dinamicamente chi fa cosa e in che ordine.
- **Event-Driven:** L'esecuzione è guidata da eventi (es. "Quando il file viene salvato, attiva l'agente revisore").

Deep Dive: Sviluppare con CrewAI

CrewAI è attualmente uno dei framework più popolari per costruire sistemi multi-agente. Si basa sul concetto di "Role-playing".

Approccio CLI-First: CrewAI incoraggia l'uso della sua CLI per generare la struttura del progetto (`crewai create crew my_project`).

Questo crea una struttura pulita che separa nettamente la configurazione dei prompt dalla logica di orchestrazione.



La Regola dell'80/20 nel Design degli Agenti

Un errore comune è spendere ore a perfezionare il prompt dell'agente, trascurando il task.

La regola d'oro di CrewAI: L'80% del tuo sforzo dovrebbe andare nella progettazione dei *Task*, e solo il 20% nella definizione degli *Agenti*.

Anche l'agente meglio definito fallirà se il task è ambiguo. Al contrario, un task ben progettato può far eccellere anche un agente semplice.

- **Evita i "God Tasks":** Non chiedere a un agente di "Cercare trend, analizzare competitor, creare un piano e fare un grafico" in un solo task.
- **Single Purpose:** Dividi il lavoro in task sequenziali, atomici e focalizzati su un singolo output chiaro.

II Framework Role-Goal-Backstory (1/2)

In CrewAI, un agente è definito da tre pilastri fondamentali (configurati in `agents.yaml`). I primi due definiscono *chi è* e *cosa deve ottenere*.

1. Role (Ruolo): La funzione specializzata dell'agente.

- ❌ **Sbagliato:** "Scrittore" (*Troppo generico, l'LLM userà uno stile standard e piatto*).
- ✅ **Giusto:** "Technical Blog Writer specializzato in AI per un'audience non tecnica".

2. Goal (Obiettivo): Lo scopo finale e i criteri di successo.

- ❌ **Sbagliato:** "Scrivere un articolo sull'AI". (*Manca di direzione e misurabilità*).
- ✅ **Giusto:** "Creare contenuti accurati e coinvolgenti che spieghino concetti complessi di AI in modo accessibile, portando il lettore a iscriversi alla newsletter".

II Framework Role-Goal-Backstory (2/2)

Il terzo pilastro dà "colore", tono di voce e profondità alle decisioni dell'agente.

3. Backstory (Background): L'esperienza, la prospettiva e i bias (positivi) dell'agente.

- ❌ **Sbagliato:** "Sei un esperto di tecnologia."
(Non aggiunge alcun dettaglio utile al comportamento).
- ✅ **Giusto:** "Hai 10 anni di esperienza nella divulgazione scientifica. Hai lavorato per testate come Wired. Credi fermamente che la chiarezza venga prima del gergo tecnico e odi gli articoli noiosi. Il tuo tono è sempre entusiasta ma rigoroso."

RPG AUTOMATION AGENT // CHARACTER PROFILE



ROLE: Process Execution Specialist

Primary function: Execute automated workflows with precision and efficiency. Interacts with digital systems, manages data flow, and ensures task completion according to defined parameters.



GOAL: Optimize Operational Efficiency

Short-term: Complete assigned automation cycles with <0.1% error rate.

Long-term: Achieve autonomous process improvement and seamless system integration.



BACKSTORY: Developed in the corporate labs of Apex Automata Solutions. Initially programmed for data migration, 'Unit-734' evolved through iterative learning algorithms, gaining proficiency in complex decision-making within structured environments. Recommissioned for general automation.

STATS



SPEED: 95



PRECISION: 99



ADAPTABILITY: 80



PROCESSING: 90

SYSTEM: AUTONOMOUS PROCESS AGENT - V3.0

Separazione delle Responsabilità: YAML

CrewAI promuove la separazione tra i prompt (YAML) e l'orchestrazione (Python). Questo permette a figure non tecniche (es. Prompt Engineer) di modificare il comportamento degli agenti senza toccare il codice.

tasks.yaml :

```
research_task:  
  description: "Ricerca i top 5 trend AI per il 2024."  
  expected_output: "Una lista markdown dei 5 trend."  
  agent: researcher
```

Separazione delle Responsabilità: Python

Il codice Python si limita a caricare la configurazione YAML e a orchestrare l'esecuzione, mantenendo il progetto pulito e manutenibile.

crew.py :

```
from crewai import Task

@task
def research_task(self) -> Task:
    # Carica la configurazione dal file YAML
    return Task(config=self.tasks_config['research_task'])
```

CrewAI Flows: Oltre la Sequenzialità

Le "Crews" sono ottime per task collaborativi, ma a volte serve logica procedurale (if/else, loop, chiamate API dirette).

I **Flows** permettono di costruire architetture *Event-Driven* combinando codice Python, chiamate LLM dirette e Crews.

Si basano su decoratori:

- `@start()` : Il punto di ingresso del flusso.
- `@listen(step_precedente)` : Si attiva quando la funzione specificata termina.
- `@router()` : Per creare percorsi condizionali (if/else).

```
class MyFlow(Flow):  
    @start()  
    def get_input(self):  
        return "AI Trends"  
  
    @listen(get_input)  
    def run_crew(self, topic):  
        # Esegue la crew passando il topic  
        return MyCrew().crew().kickoff(  
            inputs={'topic': topic}  
        )
```

Gestione dello Stato nei Flows

I Flows offrono una gestione dello stato integrata per condividere dati tra i vari step.

Stato Strutturato (Pydantic): Definisci uno schema rigido per lo stato, ottenendo type-safety e autocompletamento.

I Flows supportano anche il decoratore `@persist()` per salvare automaticamente lo stato su disco/DB, rendendo il flusso "resumable" in caso di crash.

```
class AppState(BaseModel):
    topic: str = ""
    is_approved: bool = False

class ResearchFlow(Flow[AppState]):
    @start()
    def init_state(self):
        self.state.topic = "Agenti AI"

    @listen(init_state)
    def do_research(self):
        # Usa self.state.topic
        print(f"Ricerca su: {self.state.topic}")
```

smolagents: Il Paradigma Code-Driven

Sviluppato da Hugging Face, **smolagents** è un framework minimalista (~1000 righe di codice) che sfida lo standard del mercato.

Il problema degli altri framework: Quasi tutti usano JSON per chiamare i tool. Ma il JSON non è fatto per programmare. Non puoi fare cicli (loop), non puoi salvare variabili complesse (es. un'immagine generata), non puoi annidare chiamate.

La soluzione: Code Agents In `smolagents`, l'agente scrive ed esegue direttamente **codice Python**.

- **Componibilità:** Può combinare tool, usare if/else e cicli for.
- **Gestione Oggetti:** Può salvare l'output di un tool in una variabile e passarlo a un altro.
- **Naturale per l'LLM:** I modelli sono stati addestrati su terabyte di codice Python, sanno già come usarlo perfettamente!

1. CodeAgent (Il Programmatore)

Il paradigma principale di `smolagents`.

- **Come agisce:** Genera snippet di codice Python invece di JSON.
- **Esecuzione:** Il codice viene eseguito (in locale o in sandbox).
- **Pro:** Altamente espressivo. Ideale per ragionamenti multi-step, manipolazione dati, e logica dinamica (if/else, cicli for).
- **Contro:** Rischio di errori di sintassi, richiede obbligatoriamente un ambiente di esecuzione sicuro.

2. ToolCallingAgent (II Dispatcher)

L'approccio classico, supportato anche da `smolagents` per retrocompatibilità.

- **Come agisce:** Genera chiamate strutturate in JSON (standard OpenAI).
- **Esecuzione:** Nessun codice eseguito, solo parsing del JSON e invocazione della funzione Python sottostante.
- **Pro:** Estremamente affidabile, sicuro (nessun codice arbitrario), facile da validare.
- **Contro:** Bassa espressività. Non può combinare dinamicamente i risultati o fare ragionamenti complessi tra un tool e l'altro.

Anatomia di un Tool in smolagents

Un tool non è solo una funzione, è un'interfaccia per l'LLM. Per funzionare, ha bisogno di **metadati** che verranno iniettati nel System Prompt:

- **Nome:** Chiaro e descrittivo (es. `model_download_tool`).
- **Descrizione:** Un vero e proprio "manuale di istruzioni" per l'LLM.
- **Input/Output:** Tipi e descrizioni degli argomenti.

Due modi per crearli:

1. **Decoratore** `@tool` : Per funzioni semplici.
2. **Sottoclasse** `Tool` : Per tool complessi che necessitano di stato interno o metodi aggiuntivi.

Esempio: Creare un Tool con @tool

```
from smolagents import tool

@tool
def get_weather(location: str) -> str:
    """
    Ottiene il meteo attuale.

    Args:
        location: La città (es. 'Roma', 'Milano').
    """
    return "Soleggiato, 25°C"
```

Il framework estrae automaticamente i tipi e la docstring per istruire l'agente!

L'Ecosistema dei Tool

`smolagents` è agnostico rispetto ai tool e si integra con l'intero ecosistema AI:

- **Hugging Face Hub:** Puoi condividere i tuoi tool (`push_to_hub`) o scaricare tool della community (`load_tool`). Puoi persino usare uno Space Gradio come tool (`Tool.from_space`)!
- **LangChain:** Puoi importare qualsiasi tool dall'immenso ecosistema LangChain (`Tool.from_langchain`).

- **MCP (Model Context Protocol):** Supporto nativo per i server MCP. Permette di usare tool standardizzati con output strutturati (JSON Schema), rendendo l'agente consapevole della struttura dei dati prima ancora di chiamare il tool.

```
from smolagents import MCPClient, CodeAgent

# Connessione a un server MCP locale
with MCPClient(server_params) as tools:
    agent = CodeAgent(tools=tools, model=model)
    agent.run("Che tempo fa a Parigi?")
```

smolagents: Sicurezza ed Esecuzione

Eseguire codice generato da LLM comporta rischi. `smolagents` affronta il problema su più livelli:

1. **LocalPythonExecutor:** Un interprete custom basato su AST. Blocca import non autorizzati, limita le operazioni per evitare loop infiniti e vieta l'accesso a sottomoduli pericolosi.
2. **Sandboxing Remoto:** Per la massima sicurezza, il codice può essere eseguito in ambienti isolati tramite `executor_type` :
 - **E2B / Blaxel / Modal:** Macchine virtuali o container cloud che si avviano in millisecondi.
 - **Docker / Wasm:** Esecuzione isolata in locale.

Esempio: Esecuzione in Sandbox (smolagents)

```
from smolagents import CodeAgent, InferenceClientModel

# Esecuzione sicura in sandbox E2B
agent = CodeAgent(
    model=InferenceClientModel(),
    tools=[],
    executor_type="e2b"
)

# L'agente scriverà ed eseguirà codice Python in un ambiente isolato
agent.run("Calcola il 100esimo numero di Fibonacci")
```

smolagents: Gestione della Memoria

La memoria dell'agente (`agent.memory`) registra ogni step e ogni snippet di codice eseguito.

È possibile:

- **Ispezionarla** per fare debug e capire il ragionamento dell'agente.
- **Fare il replay** (`agent.replay()`) per riavvolgere lo stato a un punto precedente.
- **Modificarla dinamicamente** tramite `step_callbacks` (es. per rimuovere immagini vecchie o log troppo lunghi e risparmiare token durante task complessi).

smolagents: Telemetria e Ispezione

Il debug di agenti multi-step è complesso. `smolagents` supporta **OpenTelemetry**. Con poche righe di codice, puoi inviare i log a piattaforme come **Langfuse** o **Arize Phoenix** per visualizzare l'intero processo di ragionamento.

```
from openinference.instrumentation.smolagents import SmolagentsInstrumentor
from langfuse import get_client

# Inizializza la telemetria
SmolagentsInstrumentor().instrument()

# Esegui l'agente normalmente
agent.run("Task complesso...")

# I log appariranno sulla dashboard di Langfuse!
```

Atomic Agents: Il Paradigma Modulare

Atomic Agents adotta un approccio ispirato ai principi di progettazione software (Single Responsibility Principle).

- Invece di grandi agenti "umani", crei piccoli agenti "atomici" altamente specializzati.
- Il cuore del framework è l'uso di **Pydantic** e della libreria **Instructor** per garantire che ogni input e output sia rigorosamente tipizzato.
- **SystemPromptGenerator**: Permette di strutturare il prompt in modo modulare definendo `background`, `steps` e `output_instructions`, mantenendo il codice pulito.

Esempio: Agente Fortemente Tipizzato

```
from atomic_agents import AtomicAgent, BaseIOSchema
from atomic_agents.context import SystemPromptGenerator

class ChatInput(BaseIOSchema):
    chat_message: str

class ChatOutput(BaseIOSchema):
    response: str
    suggested_questions: list[str]

# L'agente è fortemente tipizzato sia in input che in output!
agent = AtomicAgent[ChatInput, ChatOutput](
    config=AgentConfig(
        client=client,
        model="gpt-4o",
        system_prompt_generator=SystemPromptGenerator(
            background=["Sei un assistente esperto."],
            steps=["Analizza", "Rispondi", "Suggerisci"]
        )
    )
)
```

Atomic Agents: ChatHistory e Turni

ChatHistory e il Ciclo dei Turni: La memoria è gestita automaticamente. Ogni chiamata ad `agent.run()` crea un "Turno" (con un UUID univoco) che raggruppa il messaggio dell'utente e la risposta dell'assistente.

Puoi monitorare i token esatti in ogni momento con `agent.get_context_token_count()`.

La `chatHistory` può essere serializzata (`history.dump()`) e ricaricata (`history.load()`) per persistere le conversazioni su un database.

Atomic Agents: Dynamic Context Providers

A differenza della cronologia chat, i Context Providers iniettano informazioni dinamiche direttamente nel *System Prompt* ad ogni esecuzione. Ideali per Dati RAG o info di sessione.

```
class UserContextProvider(BaseDynamicContextProvider):
    def get_info(self) -> str:
        return f"Utente loggato: {self.user_name}"

# Registrazione del provider
agent.register_context_provider("user", UserContextProvider())

# Esecuzione (gestisce automaticamente la history)
response = agent.run(ChatInput(chat_message="Ciao!"))

# Quanti token stiamo usando?
tokens = agent.get_context_token_count()
print(f"Token totali: {tokens.total}")
```

Pattern di Memoria Multi-Agente

In Atomic Agents, puoi orchestrare la memoria tra più agenti in vari modi:

1. **Shared History:** Più agenti condividono la stessa istanza di `ChatHistory`.
2. **Independent Histories:** Ogni agente ha la sua memoria isolata.
3. **Agent-to-Agent Messaging:** L'output strutturato dell'Agente A viene iniettato esplicitamente nella memoria dell'Agente B.
4. **Supervisor-Worker:** Usano un *Context Provider* condiviso come "lavagna" per aggiornare lo stato dei task.

Esempio: Agent-to-Agent Messaging

```
# Il revisore valuta il testo e restituisce un output strutturato
review = reviewer.run(ReviewerInput(content=testo))

if not review.approved:
    # Passiamo il feedback esplicitamente alla memoria dello scrittore
    writer.history.add_message(
        "user",
        WriterInput(task=f"Correggi: {review.feedback}")
    )

# Lo scrittore riprova (senza input diretto, usa la history)
nuovo_testo = writer.run()
```

Atomic Forge e Gestione dei Tool

La Filosofia "Atomic Forge": A differenza di altri framework che nascondono i tool in librerie opache, Atomic Agents usa un approccio modulare. I tool vengono scaricati nel tuo progetto tramite CLI (`atomic`). *Vantaggio:* Hai il **controllo totale** sul codice e sulle dipendenze del tool.

Struttura Rigorosa: Ogni tool deve definire:

- `BaseIOSchema` per Input e Output.
- `BaseToolConfig` per le configurazioni (es. API keys).
- Una classe principale che eredita da `BaseTool` .

Esempio: Esecuzione di un Tool (Atomic)

```
# Inizializzazione di un tool scaricato in locale
calculator = CalculatorTool(
    config=CalculatorToolConfig()
)

# Esecuzione tipizzata e validata
result = calculator.run(
    CalculatorInputSchema(expression="2 + 2")
)
print(result.value) # 4
```

Se vuoi modificare come il calcolatore gestisce gli errori, puoi farlo direttamente nel file scaricato nel tuo repository!

Hooks System per Controllo e Retry

Atomic Agents include un potente sistema di **Hooks** (eventi) a zero overhead per monitorare e intervenire nel ciclo di vita dell'agente.

Eventi Supportati:

- `parse:error` : Fallimento validazione Pydantic.
- `completion:kwargs` : Prima della chiamata API.
- `completion:response` : Dopo la risposta API.
- `completion:error` : Errori di rete o API.

Casi d'uso: Logging strutturato in produzione, calcolo del tempo di risposta medio, o strategie di **Retry Intelligenti** (es. backoff esponenziale se si riceve un errore 429 Rate Limit).

Esempio: Implementazione degli Hook

```
def on_parse_error(error):  
    # Estrazione dettagliata dell'errore Pydantic  
    print(f"Errore nel campo: {error.errors()[0]['loc']}")  
  
def on_api_error(error, **kwargs):  
    if "rate limit" in str(error).lower():  
        print("Rate limit! Attendo e riprovo...")  
        # Logica di retry...  
  
# Registrazione degli hook  
agent.register_hook("parse:error", on_parse_error)  
agent.register_hook("completion:error", on_api_error)
```